

EDA_Supersales_store(09/09/2026)

```
In [112]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [113]: import os
os.getcwd()
import pandas as pd

df = pd.read_csv(r"samplesuperstore.csv")
print("Dataset loaded successfully")
```

Dataset loaded successfully

```
In [114]: # shows full dataframe or dataset records with all colums
df
```

Out[114]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region
0	1	US-2023-103800	1/3/2023	1/7/2023	Standard Class	DP-13000	Darren Powers	Consumer	United States
1	2	US-2023-112326	1/4/2023	1/8/2023	Standard Class	PO-19195	Phillina Ober	Home Office	United States
2	3	US-2023-112326	1/4/2023	1/8/2023	Standard Class	PO-19195	Phillina Ober	Home Office	United States
3	4	US-2023-112326	1/4/2023	1/8/2023	Standard Class	PO-19195	Phillina Ober	Home Office	United States
4	5	US-2023-141817	1/5/2023	1/12/2023	Standard Class	MB-18085	Mick Brown	Consumer	United States
...	...	...	...	...	...	...	...	...	...
10189	10190	US-2026-143259	12/30/2026	1/3/2027	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States
10190	10191	US-2026-115427	12/30/2026	1/3/2027	Standard Class	EB-13975	Erica Bern	Corporate	United States
10191	10192	US-2026-156720	12/30/2026	1/3/2027	Standard Class	JM-15580	Jill Matthias	Consumer	United States
10192	10193	US-2026-143259	12/30/2026	1/3/2027	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States
10193	10194	CA-2026-143500	12/30/2026	1/3/2027	Standard Class	HO-15230	Harry Olson	Consumer	Canada

10194 rows × 21 columns

```
In [115]: # First 5 rows
df.head()
```

Out[115]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	
0	1	US-2023-103800	1/3/2023	1/7/2023	Standard Class	DP-13000	Darren Powers	Consumer	United States	
1	2	US-2023-112326	1/4/2023	1/8/2023	Standard Class	PO-19195	Phillina Ober	Home Office	United States	
2	3	US-2023-112326	1/4/2023	1/8/2023	Standard Class	PO-19195	Phillina Ober	Home Office	United States	
3	4	US-2023-112326	1/4/2023	1/8/2023	Standard Class	PO-19195	Phillina Ober	Home Office	United States	
4	5	US-2023-141817	1/5/2023	1/12/2023	Standard Class	MB-18085	Mick Brown	Consumer	United States	P

5 rows × 21 columns

```
In [116]: # Last 5 rows
df.tail()
```

Out[116]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	
10189	10190	US-2026-143259	12/30/2026	1/3/2027	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States	
10190	10191	US-2026-115427	12/30/2026	1/3/2027	Standard Class	EB-13975	Erica Bern	Corporate	United States	
10191	10192	US-2026-156720	12/30/2026	1/3/2027	Standard Class	JM-15580	Jill Matthias	Consumer	United States	
10192	10193	US-2026-143259	12/30/2026	1/3/2027	Standard Class	PO-18865	Patrick O'Donnell	Consumer	United States	

```
In [117]: # Random 5 rows
df.sample(5)
```

Out[117]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region
9686	9687	US-2026-117702	11/28/2026	12/4/2026	Standard Class	LS-16975	Lindsay Shagiari	Home Office	United States
2576	2577	US-2024-141768	5/25/2024	5/27/2024	Second Class	NP-18685	Nora Pelletier	Home Office	United States
8070	8071	US-2026-133249	7/8/2026	7/11/2026	First Class	SZ-20035	Sam Zeldin	Home Office	United States
6755	6756	US-2025-109827	12/25/2025	1/1/2026	Standard Class	LW-16825	Laurel Workman	Corporate	United States
4202	4203	US-2025-137295	1/8/2025	1/13/2025	Standard Class	VS-21820	Vivek Sundaresam	Consumer	United States

5 rows × 21 columns

Dataset Dimensions

```
In [118]: print("Dataset shape:",df.shape)
print("Number of Rows:",df.shape[0])
print("Number of columns:",df.shape[1])
print("Total Elements:",df.size)

Dataset shape: (10194, 21)
Number of Rows: 10194
Number of columns: 21
Total Elements: 214074
```

```
In [119]: # Columns name
df.columns
```

Out[119]:

Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country/Region', 'City', 'State/Province', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'], dtype='str')

In [120]: `df.dtypes`

```
Out[120]: Row ID          int64
Order ID          str
Order Date        str
Ship Date         str
Ship Mode         str
Customer ID       str
Customer Name     str
Segment          str
Country/Region    str
City             str
State/Province    str
Postal Code       str
Region           str
Product ID       str
Category         str
Sub-Category     str
Product Name     str
Sales            float64
Quantity         int64
Discount         float64
Profit           float64
dtype: object
```

In [121]: `df.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 10194 entries, 0 to 10193
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Row ID                10194 non-null  int64
 1   Order ID              10194 non-null  str
 2   Order Date            10194 non-null  str
 3   Ship Date             10194 non-null  str
 4   Ship Mode             10194 non-null  str
 5   Customer ID           10194 non-null  str
 6   Customer Name         10194 non-null  str
 7   Segment               10194 non-null  str
 8   Country/Region        10194 non-null  str
 9   City                  10194 non-null  str
10   State/Province        10194 non-null  str
11   Postal Code           10194 non-null  str
12   Region                10194 non-null  str
13   Product ID            10194 non-null  str
14   Category              10194 non-null  str
15   Sub-Category          10194 non-null  str
16   Product Name          10194 non-null  str
17   Sales                 10194 non-null  float64
18   Quantity              10194 non-null  int64
19   Discount              10194 non-null  float64
20   Profit                10194 non-null  float64
dtypes: float64(3), int64(2), str(16)
memory usage: 1.6 MB
```

Statistical summary

In [122]: `df.describe()`

Out[122]:

	Row ID	Sales	Quantity	Discount	Profit
count	10194.000000	10194.000000	10194.000000	10194.000000	10194.000000
mean	5097.500000	228.225854	3.791838	0.155385	28.673417
std	2942.898656	619.906839	2.228317	0.206249	232.465115
min	1.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2549.250000	17.220000	2.000000	0.000000	1.760800
50%	5097.500000	53.910000	3.000000	0.200000	8.690000
75%	7645.750000	209.500000	5.000000	0.200000	29.297925
max	10194.000000	22638.480000	14.000000	0.800000	8399.976000

Dataset understanding

In [123]: `# Numerical colmns`
`numerical_columns = df.select_dtypes(include=np.number).columns`
`print("Numerical Columns:")`
`print(numerical_columns)`

Numerical Columns:
 Index(['Row ID', 'Sales', 'Quantity', 'Discount', 'Profit'], dtype='str')

In [124]: `# Categorical columns`
`categorical_columns = df.select_dtypes(include='str').columns`
`print(categorical_columns)`

Index(['Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID',
 'Customer Name', 'Segment', 'Country/Region', 'City', 'State/Province',
 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',
 'Product Name'],
 dtype='str')

Missing Values

```
In [125]: df.isnull().sum()
```

```
Out[125]: Row ID          0
Order ID          0
Order Date        0
Ship Date         0
Ship Mode         0
Customer ID       0
Customer Name     0
Segment          0
Country/Region    0
City              0
State/Province    0
Postal Code       0
Region           0
Product ID        0
Category          0
Sub-Category      0
Product Name      0
Sales             0
Quantity          0
Discount          0
Profit            0
dtype: int64
```

removing values

```
In [126]: # Numbers of rows before removing null values
print("Rows before removing null values:",df.shape[0])

# Remove rows containing null values
df = df.dropna()

# Number of rows after removing null values
print("Rows after removing null values:",df.shape[0])

Rows before removing null values: 10194
Rows after removing null values: 10194
```

```
In [127]: print("Number of duplicate rows:",df.duplicated().sum())

Number of duplicate rows: 0
```

```
In [128]: df = df.drop_duplicates()

print("Dataset shape after removing duplicates:",df.shape)

Dataset shape after removing duplicates: (10194, 21)
```

```
In [129]: # data cleaning and preprocessing
df["Order date"] = pd.to_datetime(df["Order Date"])
df["Ship date"] = pd.to_datetime(df["Ship Date"])
df[["Order date", "Ship date"]].dtypes
```

```
Out[129]: Order date      datetime64[us]
Ship date      datetime64[us]
dtype: object
```

```
In [130]: text_columns = ["Category", "Sub-Category", "Segment"]

for column in text_columns:
    df[column]=(
        df[column]
        .astype(str)
        .str.strip()
        .str.title()
    )
```

```
In [131]: for column in text_columns:
            print("\n,column")
            print(df[column].unique())
```

```
,column
<StringArray>
['Office Supplies', 'Furniture', 'Technology']
Length: 3, dtype: str
```

```
,column
<StringArray>
[ 'Paper',      'Binders',      'Labels',      'Storage',      'Ar
t',
  'Chairs',      'Fasteners',      'Phones', 'Furnishings', 'Accessorie
s',
  'Bookcases',   'Envelopes',   'Appliances',      'Tables',      'Supplie
s',
  'Machines',    'Copiers']
Length: 17, dtype: str
```

```
,column
<StringArray>
['Consumer', 'Home Office', 'Corporate']
Length: 3, dtype: str
```

```
In [132]: # Final data check
print("Final shape:", df.shape)

print("\nTotal Missing Values:", df.isnull().sum().sum())

print("Total Duplicate Rows:", df.duplicated().sum())

print("\nData Types:")
print(df.dtypes)
```

Final shape: (10194, 23)

Total Missing Values: 0

Total Duplicate Rows: 0

Data Types:

Row ID	int64
Order ID	str
Order Date	str
Ship Date	str
Ship Mode	str
Customer ID	str
Customer Name	str
Segment	str
Country/Region	str
City	str
State/Province	str
Postal Code	str
Region	str
Product ID	str
Category	str
Sub-Category	str
Product Name	str
Sales	float64
Quantity	int64
Discount	float64
Profit	float64
Order date	datetime64[us]
Ship date	datetime64[us]
dtype:	object

```
In [133]: numerical_features = [
            "Sales",
            "Quantity",
            "Discount",
            "Profit"
        ]

df[numerical_features].describe()
```

Out[133]:

	Sales	Quantity	Discount	Profit
count	10194.000000	10194.000000	10194.000000	10194.000000
mean	228.225854	3.791838	0.155385	28.673417
std	619.906839	2.228317	0.206249	232.465115
min	0.444000	1.000000	0.000000	-6599.978000
25%	17.220000	2.000000	0.000000	1.760800
50%	53.910000	3.000000	0.200000	8.690000
75%	209.500000	5.000000	0.200000	29.297925
max	22638.480000	14.000000	0.800000	8399.976000

```
In [134]: summary = pd.DataFrame({
            "Mean":df[numerical_features].mean(),
            "Median":df[numerical_features].median(),
            "Minmum":df[numerical_features].min(),
            "Maximum":df[numerical_features].max(),
            "Standard Devation":df[numerical_features].std()
        })

summary
```

Out[134]:

	Mean	Median	Minmum	Maximum	Standard Devation
Sales	228.225854	53.91	0.444	22638.480	619.906839
Quantity	3.791838	3.00	1.000	14.000	2.228317
Discount	0.155385	0.20	0.000	0.800	0.206249
Profit	28.673417	8.69	-6599.978	8399.976	232.465115

Sales and profit Analysis

```
In [135]: # Category wise sale
category_sales = (
    df.groupby("Category")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

category_sales
```

```
Out[135]: Category
Technology      839893.2790
Furniture       754747.7613
Office Supplies 731893.3140
Name: Sales, dtype: float64
```

```
In [136]: # Category wise profit
category_profit = (
    df.groupby("Category")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

category_profit
```

```
Out[136]: Category
Technology      839893.2790
Furniture       754747.7613
Office Supplies 731893.3140
Name: Sales, dtype: float64
```

```
In [137]: # Sub category wise profit
sub_category_profit = (
    df.groupby("Category")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

sub_category_profit
```

```
Out[137]: Category
Technology      839893.2790
Furniture       754747.7613
Office Supplies 731893.3140
Name: Sales, dtype: float64
```

```
In [138]: # REGION WISE PROFIT
region_profit = (
    df.groupby("Region")["Profit"]
    .sum()
    .sort_values(ascending=False)
)

region_profit
```

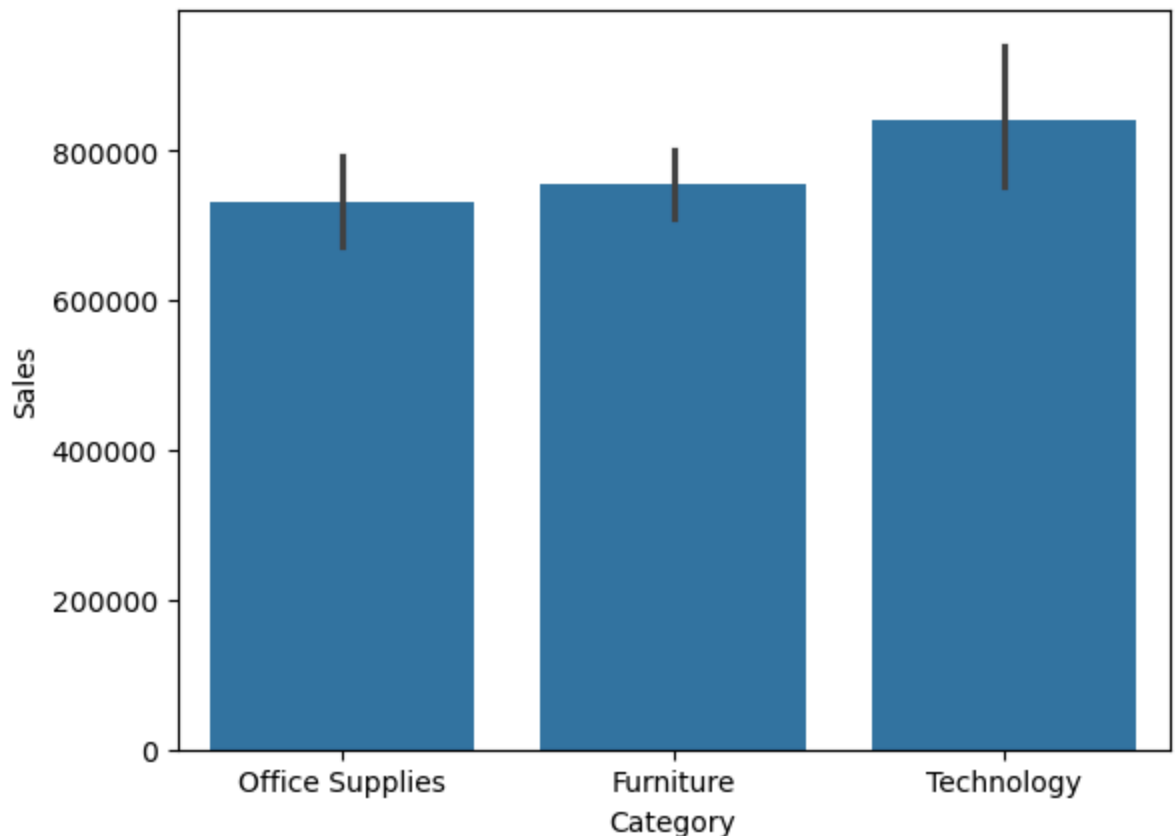
```
Out[138]: Region
West      110798.8170
East      94883.2603
South     46749.4303
Central   39865.3070
Name: Profit, dtype: float64
```

```
In [139]: plt.figure(figsize=(8, 5))
```

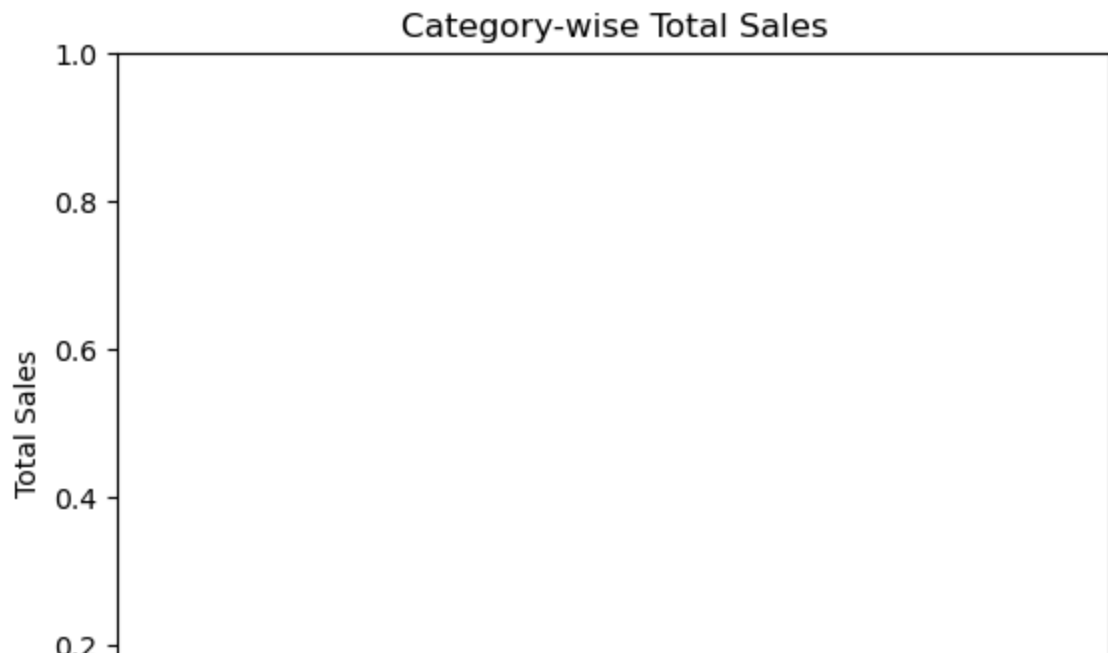
```
Out[139]: <Figure size 800x500 with 0 Axes>
<Figure size 800x500 with 0 Axes>
```

```
In [140]: sns.barplot(
    data=df,
    x="Category",
    y="Sales",
    estimator="sum"
)
```

```
Out[140]: <Axes: xlabel='Category', ylabel='Sales'>
```



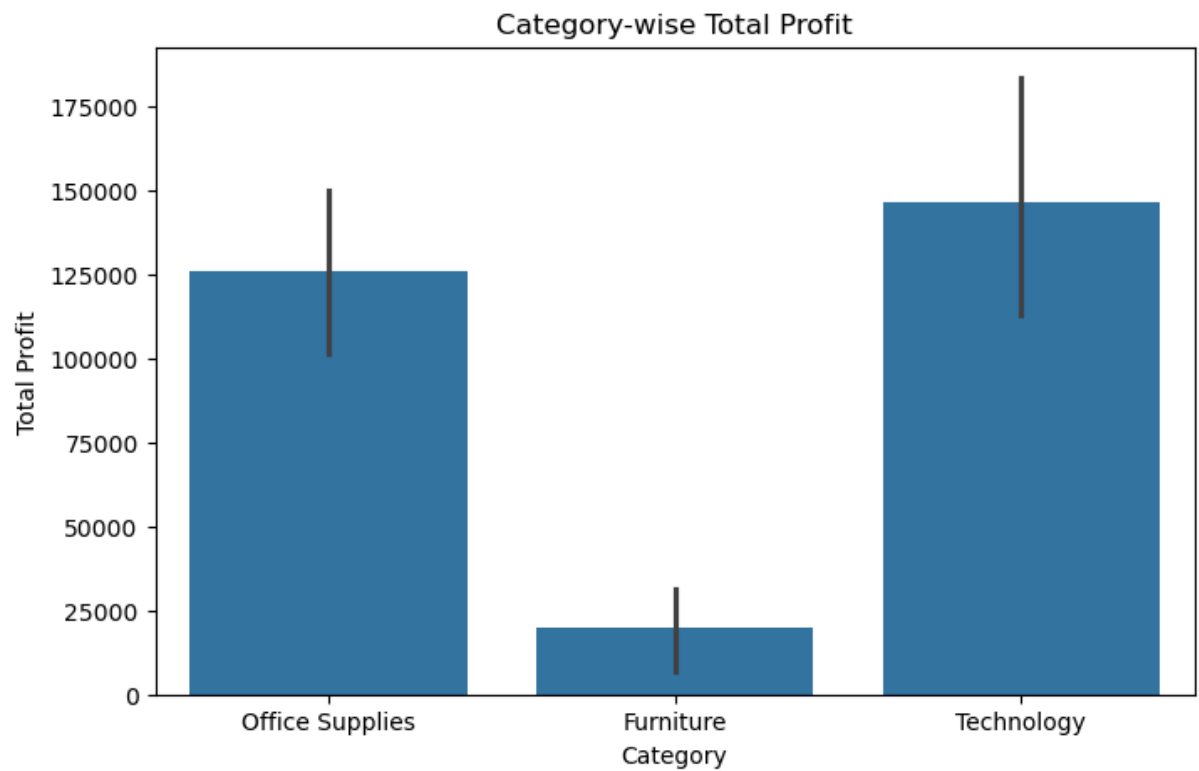
```
In [141]: plt.title("Category-wise Total Sales")  
plt.xlabel("Category")  
plt.ylabel("Total Sales")  
plt.show()
```



```
In [142]: plt.figure(figsize=(8, 5))

sns.barplot(
    data=df,
    x="Category",
    y="Profit",
    estimator="sum"
)

plt.title("Category-wise Total Profit")
plt.xlabel("Category")
plt.ylabel("Total Profit")
plt.show()
```

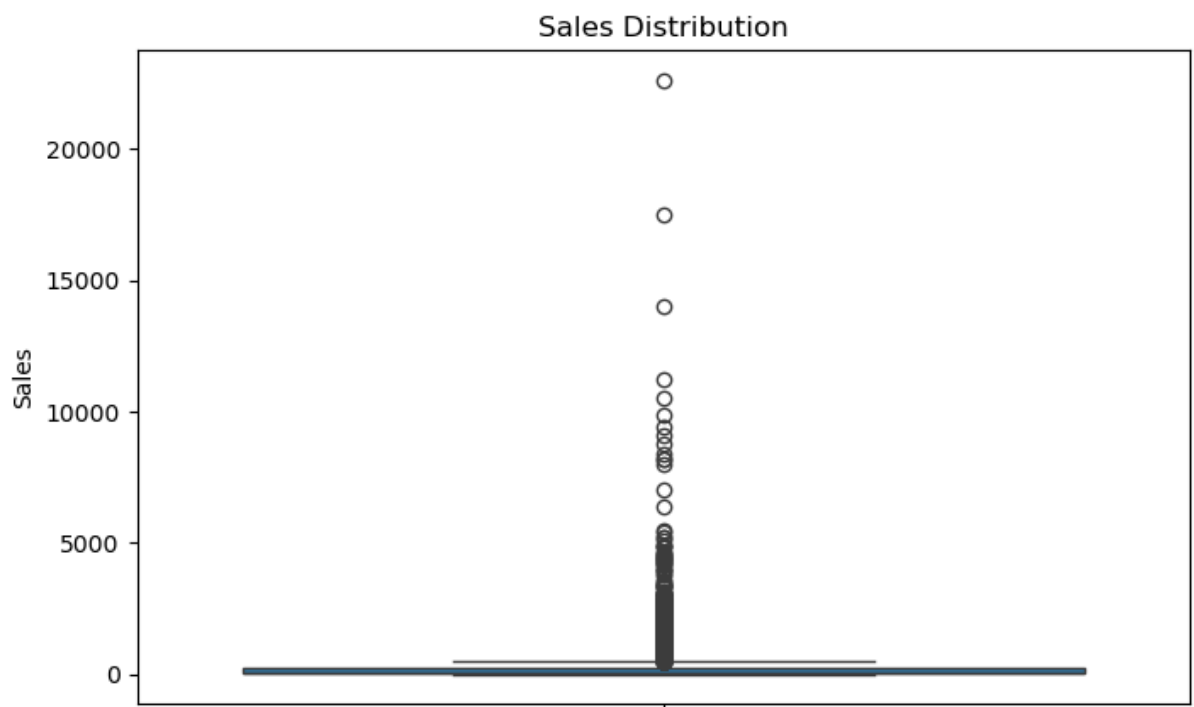


```
In [143]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    y="Sales"
)

plt.title("Sales Distribution")
plt.ylabel("Sales")

plt.show()
```



```
In [144]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    y="Profit"
)

plt.title("Profit Distribution")
plt.ylabel("Profit")

plt.show()
```

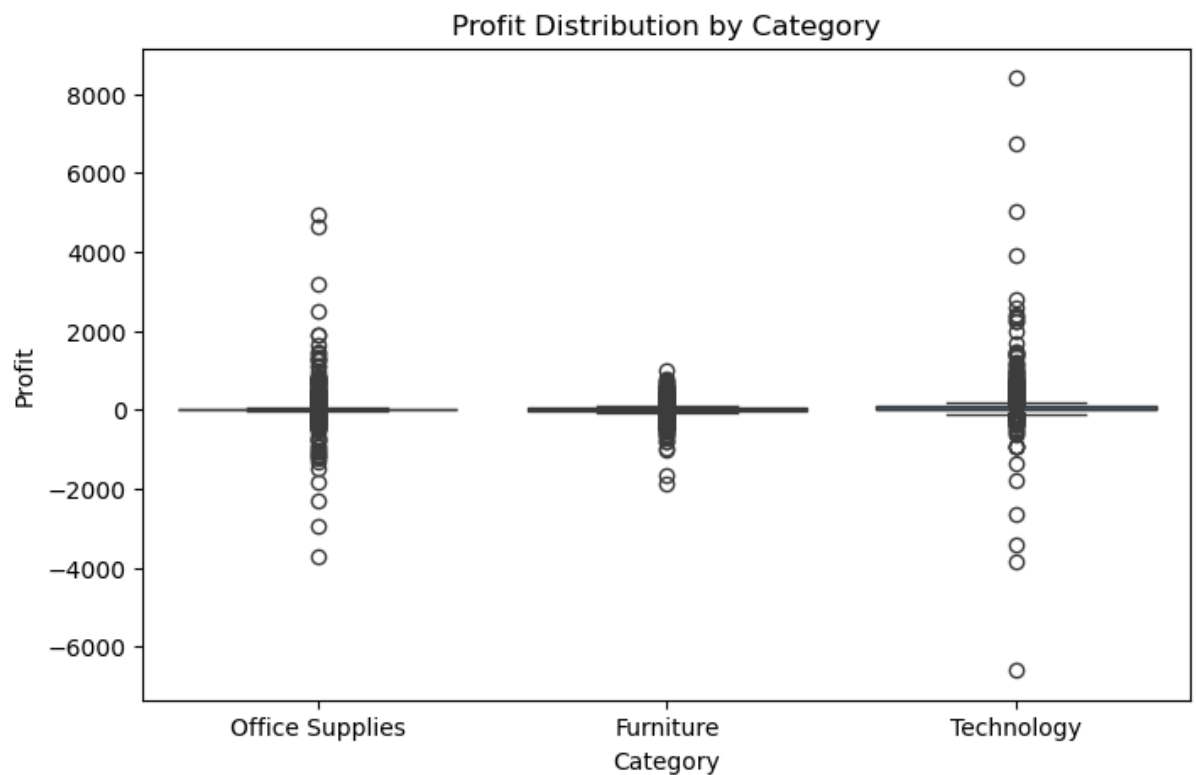


```
In [145]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="Category",
    y="Profit"
)

plt.title("Profit Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Profit")

plt.show()
```

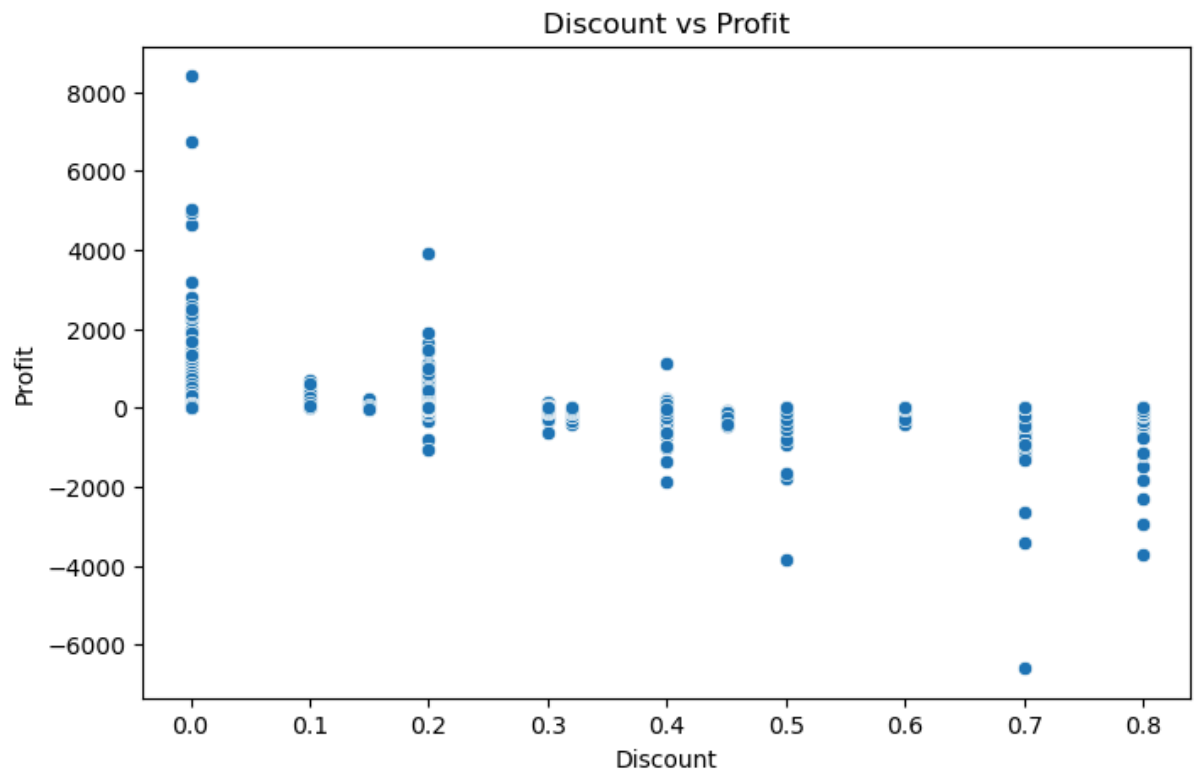


Discount vs Profit

```
In [146]: plt.figure(figsize=(8, 5))

sns.scatterplot(
    data = df,
    x = "Discount",
    y = "Profit",
)
plt.title("Discount vs Profit")
plt.xlabel("Discount")
plt.ylabel("Profit")

plt.show()
```

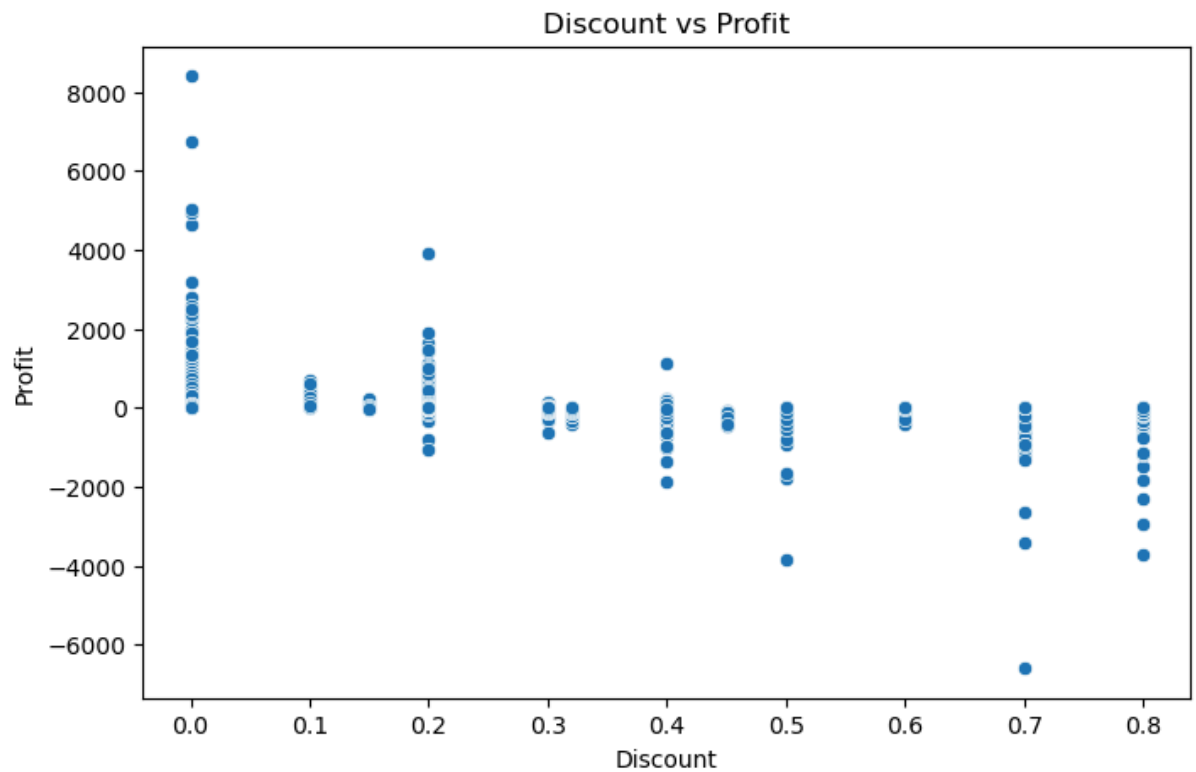


Calulating profit Margin

```
In [148]: plt.figure(figsize=(8, 5))

sns.scatterplot(
    data = df,
    x = "Discount",
    y = "Profit",
)
plt.title("Discount vs Profit")
plt.xlabel("Discount")
plt.ylabel("Profit")

plt.show()
```



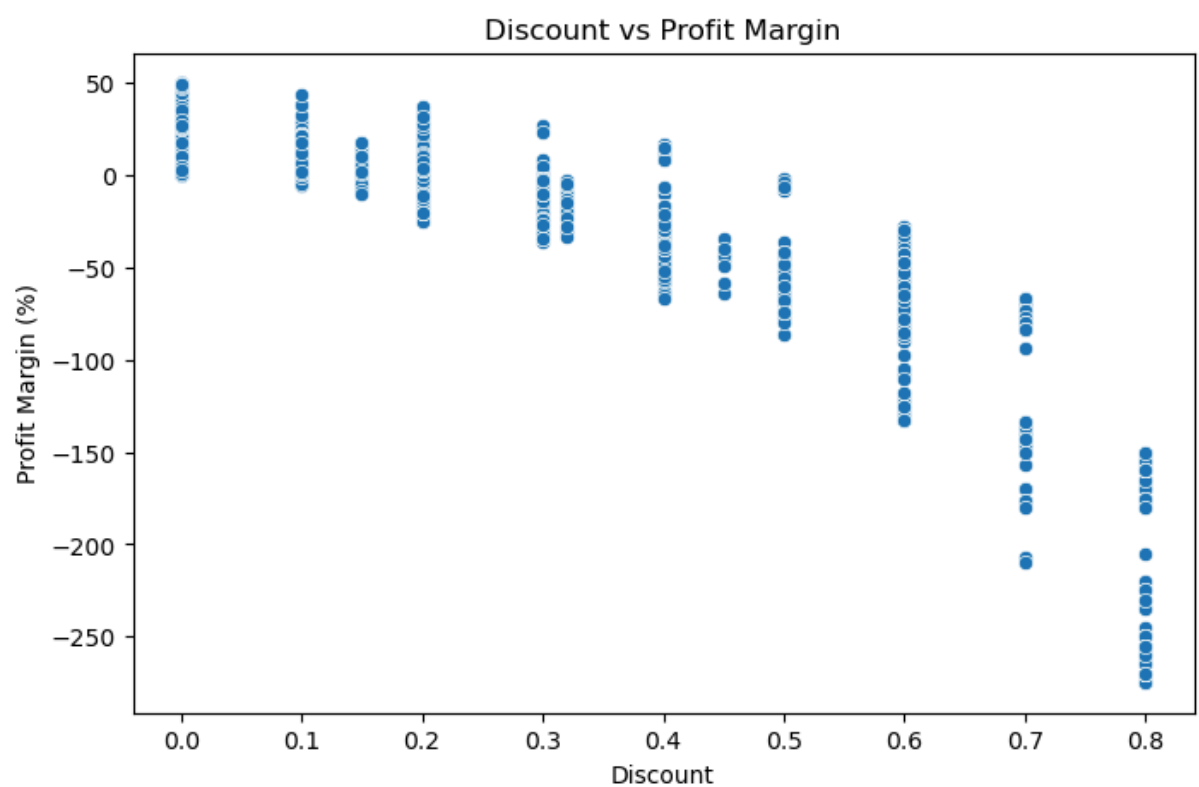
Discount vs margin profit

```
In [162]: plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="Discount",
    y="Profit Margin"
)

plt.title("Discount vs Profit Margin")
plt.xlabel("Discount")
plt.ylabel("Profit Margin (%)")

plt.show()
```



```
In [163]: numerical_df = df.select_dtypes(include=np.number)

correlation_matrix = numerical_df.corr()

correlation_matrix
```

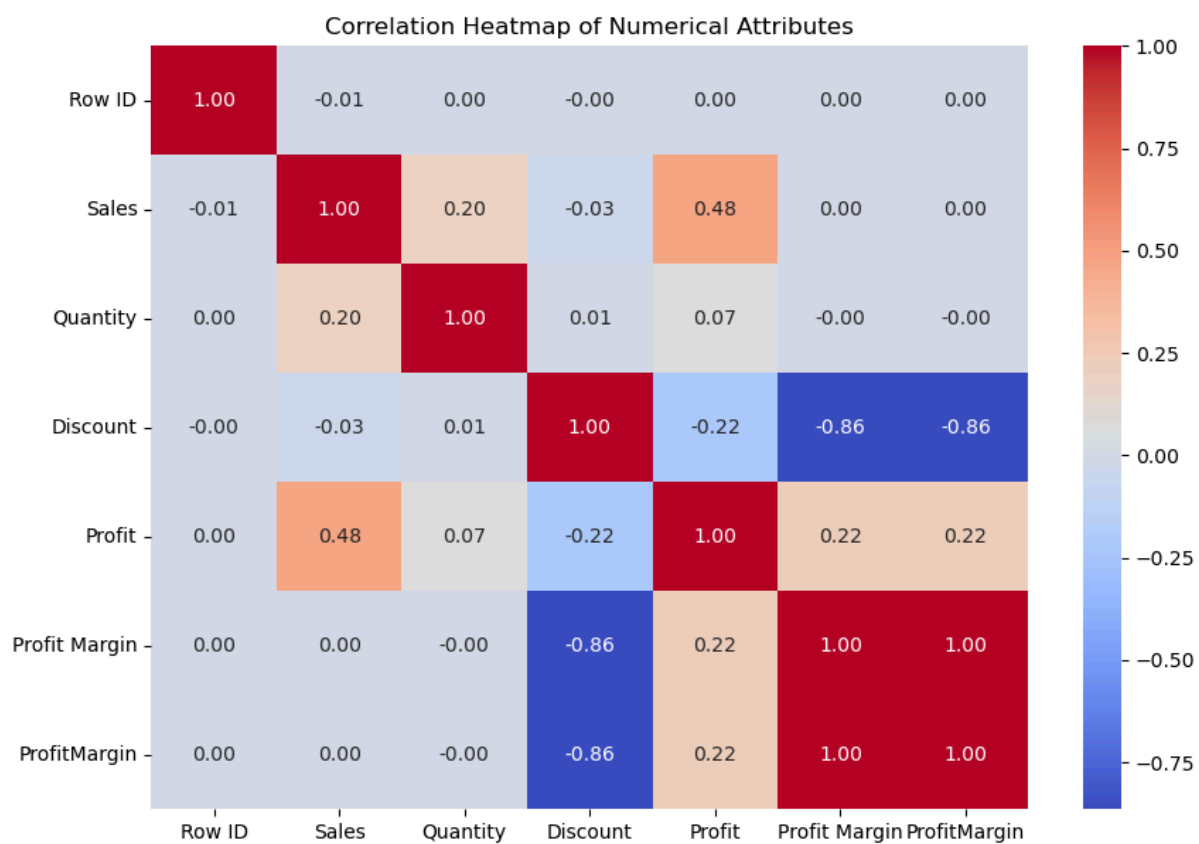
Out[163]:

	Row ID	Sales	Quantity	Discount	Profit	Profit Margin	ProfitMargin
Row ID	1.000000	-0.008958	0.000502	-0.001474	0.003022	0.001198	0.001198
Sales	-0.008958	1.000000	0.198457	-0.027756	0.481460	0.002707	0.002707
Quantity	0.000502	0.198457	1.000000	0.007475	0.066145	-0.003830	-0.003830
Discount	-0.001474	-0.027756	0.007475	1.000000	-0.218882	-0.864821	-0.864821
Profit	0.003022	0.481460	0.066145	-0.218882	1.000000	0.223224	0.223224
Profit Margin	0.001198	0.002707	-0.003830	-0.864821	0.223224	1.000000	1.000000
ProfitMargin	0.001198	0.002707	-0.003830	-0.864821	0.223224	1.000000	1.000000

```
In [164]: plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)
plt.title("Correlation Heatmap of Numerical Attributes")

plt.show()
```



```
In [ ]: nt(""" BUSINESS INSIGHTS =Questions & Answers
objective of EDA is to convert numerical results and visualizations into

Which category has the highest sales?
Technology has the highest sales.

Which category has the lowest sales?
Office Supplies has the lowest sales.

Which region contributes most to sales?
West Region.

FIT:

Which category has highest profit?
Technology

Which sub-categories have low/negative profit?
Tables

Which regions are most profitable?
West

COUNT:

Does profit decrease as discount increases?
Yes

Which discount levels have poor profitability?
40%-80% discount

Which transactions generate negative profit?
Orders with >20% discount

RELATION
Which variables have strong positive correlation?
Sales & Profit

Which variables have negative correlation?
Discount & Profit Margin.
Is discount negatively related to Profit?
Yes, proved by heatmap.
)
```

```
In [ ]: """ REPORT
Sample Superstore Sales Dataset contains 9,994 rows and 21 columns, covering
data was cleaned, and date columns were converted into datetime format. Exp
er plots, and correlation analysis to understand sales, profit, and discou
ne highest profit of $146,543,
Furniture generated relatively low profit of $19,729 despite its substant
ation analysis indicated a strong negative relationship between Discount
ally associated with lower profitability. The analysis highlights the impo
table product categories and regions.
esses should use data-driven decisions,
or discount policies, and conduct further analysis before implementing maj
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```